EB-JEPA Hackathon Field Guide

A 24-Hour Sprint Through Joint-Embedding Predictive Architectures

Basile Terver and Théo Le Pendeven
EB-JEPA Hackathon
github.com/facebookresearch/eb_jepa

Abstract

Welcome to the EB-JEPA Hackathon. Over the next 24 hours you will get hands-on with Joint-Embedding Predictive Architectures (JEPAs): self-supervised models that learn to predict in representation space rather than pixel space. This guide is your field manual. It walks you through the three working examples shipped with the `eb_jepa` library (image representation learning, video prediction, and action-conditioned world modeling with planning), shows you the small set of components you will actually edit, and hands you a curated menu of project tracks: most aimed at carrying the JEPA recipe to new continuous, high-dimensional, noisy modalities, and some at pushing the vision and robotics settings already in the codebase. It closes with the practical logistics: compute policy on the shared B200 cluster, deliverables, and how projects are judged. Read Section 1 first, skim the rest, and start running code within the hour.

1 Welcome and the 24-Hour Game Plan

This hackathon brings together roughly 100 participants in 25 teams of four, sharing 72 NVIDIA B200 GPUs for 24 hours. The goal is not a leaderboard win; it is to understand JEPAs by building with them. By the end you should be able to explain, and have modified, the three core pieces of every JEPA: an *encoder* f_θ that maps observations to representations, a *predictor* g_ϕ that predicts future or alternate representations, and a *regularizer* $\mathcal{R}$ that stops those representations from collapsing to a constant.

Why JEPAs, and why this codebase. Production JEPA stacks (I-JEPA, V-JEPA, V-JEPA 2) (Assran et al., 2023; Bardes et al., 2024; Assran et al., 2025) are powerful but large and hard to navigate, and world-model implementations often assume frozen pre-trained encoders and bespoke setups (Zhou et al., 2024a; Terver et al., 2026b). The `eb_jepa` library (Terver et al., 2026a) exists to close that gap: three self-contained examples, each trainable on a single GPU in a few hours, that take you from static-image self-supervision to temporal prediction to action-conditioned planning, all under one energy-based objective (Section 2). That makes it an ideal sandbox for a 24-hour sprint: you can train a model end to end several times, ablate a component, and actually see the effect before the clock runs out. By design `eb_jepa` is the prototyping rung of a ladder: once an idea works here, larger codebases pick it up, `jepa-wms` (Terver et al., 2026b) for planning with frozen encoders on diverse benchmarks, and `stable-worldmodel` (Maes et al., 2026) for a broad, controllable suite of world-model environments. Prototype small here; scale and validate there.

What we want you to explore. The library ships with vision and robotics modalities. The headline challenge of this hackathon is to take the *JEPA recipe to data it was not built for*: continuous, high-dimensional, noisy signals such as audio, physiological recordings, climate and sensor fields, financial or scientific time series, and point clouds. JEPAs are a natural fit here precisely because they predict in a learned latent space and discard task-irrelevant noise, rather than reconstructing every sample. Teams who prefer to stay in vision or robotics are very welcome to push the existing examples with innovative ideas instead; Section 6 offers a menu of both. Either way, the engineering surface is the same small set of swappable components, and Section 5 tells you exactly which files to touch.

Scope realistically. Twenty-four hours is short. A successful project is one clear hypothesis, tested with a clean before/after comparison, with a figure or number to show for it. It is far better to get one new modality training without collapsing and to *understand why* than to half-wire three ambitious ideas. The single most common failure mode in JEPA training is representation collapse; budget time for it, watch the variance and prediction losses (Section 2), and read the gotchas in each example walkthrough.

1.1 A suggested timeline

Table 1 is a sane default for a team of four, not a mandate. Adapt it to your sleep schedule and your project's risk. The recurring theme: get real code running in the first hour, lock your scope early, and stop *building* with several hours to spare so you can evaluate, visualize, and write up.

Table 1: **A default 24-hour plan for a team of four.** Times are elapsed hours from kickoff. "Pods" refers to splitting the team so that environment setup, code reading, and a baseline run happen in parallel rather than in series.

Phase	Hours	What to do
Ignite	0–1	Clone, install (Section 3), launch the `image_jepa` smoke test on one GPU. Skim this guide.
Orient	1–3	In parallel pods: (i) run all three examples; (ii) read the example you will build on; (iii) pick a track (Section 6).
Lock scope	3–4	Write a one-sentence hypothesis and a success metric. Sketch the data $\rightarrow$ encoder $\rightarrow$ predictor $\rightarrow$ loss path on paper.
Baseline	4–8	Get *something* training without collapsing: data loader yields the right tensor shape, loss decreases, variance stays alive.
Iterate	8–16	One change at a time. Keep the previous run as the baseline; ablate; sweep two or three hyperparameters (Section 3).
Evaluate	16–21	Lock the model. Run downstream eval / planning / probing. Make the figures. Re-run a seed for error bars if time allows.
Write up	21–24	Push the repo (GitHub Classroom) and prepare the 10-minute presentation. State the hypothesis, the result, and one thing you learned about JEPAs.

⋆ **Tip** Treat the first training run as a plumbing test, not a science experiment. Use a tiny model, a handful of epochs, and `logging.log_wandb=false` to confirm the pipeline runs end to end in minutes. Only then turn the knobs up. A pipeline that runs at hour 5 beats a beautiful idea that first executes at hour 20.

1.2 How to read this guide

Section 2 is the two-page conceptual core: the unified JEPA objective and the regularizers, enough theory to act. Section 3 gets you installed and explains the launcher and the compute policy. Section 4 walks through the three examples in the run-it / read-it / tweak-it pattern. Section 5 is the reference you will return to most: the exact extension points for encoders, predictors, regularizers, and datasets. Section 6 is the project menu. Section 7 covers compute, deliverables, and judging. For prior work, a curated per-track reading list lives in `references/README.md` at the repo root (see Section 6). Throughout, `monospaced paths` point at real files in the repository, and boxes flag **tips**, **gotchas**, and **notes** worth pausing on.

2 The JEPA Recipe in Two Pages

This section is the whole conceptual toolkit you need to start. If you remember one thing: a JEPA learns by *predicting one representation from another*, and the only reason it does not cheat by mapping everything to a constant is the regularizer. Everything else is architecture.

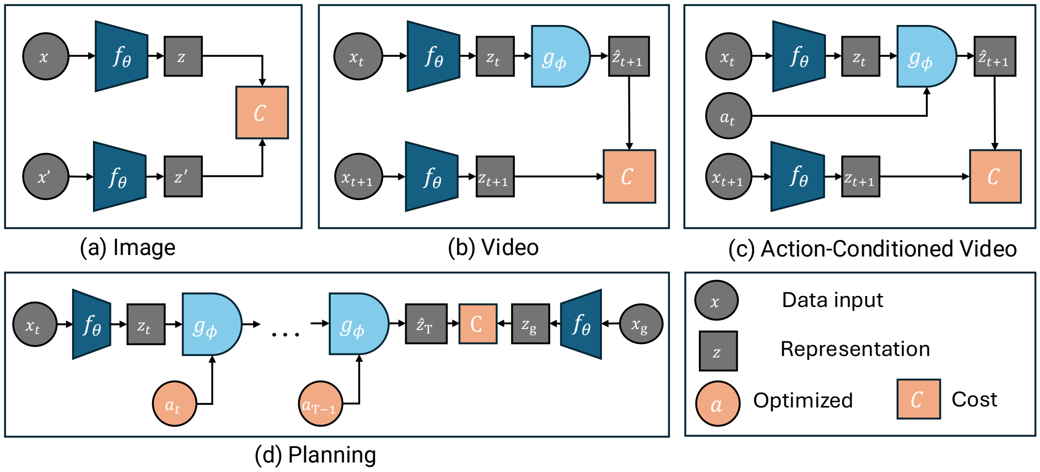

Figure 1: **One architecture, three settings.** An encoder f_θ maps inputs to representations; a predictor g_ϕ maps a context representation to a predicted target $\hat{z}$; a cost C measures prediction error in representation space. **(a)** Image: two augmented views, g_ϕ is the identity. **(b)** Video: g_ϕ predicts the next frame's representation. **(c)** Action-conditioned video: the predictor is additionally conditioned on an action a_t. **(d)** Planning: optimize an action sequence so the imagined rollout reaches a goal representation z_g.

2.1 The unified energy objective

We view JEPAs through the lens of energy-based models (LeCun et al., 2006). An energy function E assigns a scalar to an input–output pair; low energy means "compatible." For a JEPA, energy is simply prediction error in representation space. Given an encoder f_θ, a predictor g_ϕ, and optional conditioning a with its own encoder q_ω, the general energy is

$$E(x, x', a) = \mathcal{L}_{\text{pred}}\big(g_\phi(f_\theta(x), q_\omega(a)),\ f_\theta(x')\big), \tag{1}$$

and training minimizes this energy plus a regularizer $\mathcal{R}$ that prevents collapse:

$$\mathcal{L} = \mathcal{L}_{\text{pred}}\big(g_\phi(z, u), z'\big) + \lambda\,\mathcal{R}(z), \qquad z = f_\theta(x),\ u = q_\omega(a). \tag{2}$$

The three examples are Equation (2) with different choices of g_ϕ and conditioning.

(a) Image-JEPA: invariance. Two augmented views x, x' of the same image; the predictor is the identity, so the model just pulls the two representations together,

$$\mathcal{L}_{\text{image}} = \|z - z'\|_2^2 + \lambda\,\mathcal{R}(z, z'). \tag{3}$$

Low energy means the encoder has learned features invariant to the augmentation.

(b) Video-JEPA: temporal prediction. The predictor sees a context of $v{+}1$ frame representations and predicts the next one, summed over time,

$$\mathcal{L}_{\text{video}} = \sum_t \|g_\phi(z_{t-v:t}) - z_{t+1}\|_2^2 + \lambda\,\mathcal{R}(z_{1:T}). \tag{4}$$

(c) AC-video-JEPA: world modeling. An action encoder produces $u_t = q_\omega(a_{t-w:t})$ and the predictor is conditioned on it,

$$\mathcal{L}_{\text{world}} = \sum_t \|g_\phi(z_{t-v:t}, u_{t-v:t}) - z_{t+1}\|_2^2 + \lambda\,\mathcal{R}(z_{1:T}, u_{1:T}). \tag{5}$$

This is a latent dynamics model: given a state and an action, predict the next state. It is what makes planning (Section 2.4) possible.

Note. In code this is one method. `eb_jepa/jepa.py` defines a single `JEPA` module whose `unroll(...)` call handles all three settings, plus planning. Image-JEPA uses an identity predictor; video drops the action encoder; AC-video uses everything. When you build a new modality you are picking an encoder, a predictor, and a regularizer, then handing them to this same class.

2.2 Preventing collapse: the regularizers

If you only minimized prediction error, the encoder would map everything to a constant: zero error, zero information. JEPAs in this library prevent that with explicit regularization (rather than stop-gradient or EMA tricks (Grill et al., 2020; Chen & He, 2021)). Two families are implemented.

VICReg (Bardes et al., 2022) uses two terms on a batch of embeddings $Z \in \mathbb{R}^{N\times D}$. A *variance* term keeps every feature dimension from shrinking,

$$\mathcal{L}_{\text{var}}(Z) = \frac{1}{D}\sum_{j=1}^{D} \max\Big(0,\ \gamma - \sqrt{\mathrm{Var}(Z_{:,j}) + \epsilon}\Big), \tag{6}$$

with target standard deviation γ (typically 1); and a *covariance* term decorrelates dimensions so the model uses all its capacity,

$$\mathcal{L}_{\text{cov}}(Z) = \frac{1}{D(D-1)}\sum_{i\neq j}[C(Z)]_{i,j}^2, \qquad C(Z) = \frac{1}{N-1}(Z-\bar{Z})^\top(Z-\bar{Z}). \tag{7}$$

The full regularizer is $\mathcal{R}_{\text{VICReg}} = \alpha\mathcal{L}_{\text{var}} + \beta\mathcal{L}_{\text{cov}}$, computed on a learned projection $r = h_\psi(z)$ rather than directly on z.

SIGReg (Balestriero & LeCun, 2025) takes a different route. It identifies the isotropic Gaussian $\mathcal{N}(0, I)$ as the optimal embedding distribution and enforces it by testing Gaussianity along random 1D projections ξ_p,

$$\mathcal{R}_{\text{SIGReg}}(Z) = \frac{1}{P}\sum_{p=1}^{P} G(Z\xi_p), \tag{8}$$

where G is the Epps–Pulley Gaussianity statistic. It has a single hyperparameter λ and linear time and memory, which tends to make it more forgiving to tune than VICReg's two coefficients (Figure 2).

Gotcha. Watch the variance loss. If $\mathcal{L}_{\text{var}}$ shoots up and stays high while the prediction loss drops to near zero, your encoder is collapsing: the predictor is winning by making everything constant. The fix is almost always more regularization weight or a healthier projector, not less. The image example prints these per-term losses every epoch; the world-model example logs them too.

2.3 Two ingredients specific to world models

Multistep rollout. Training a predictor only one step ahead, then unrolling it autoregressively at test time, creates *exposure bias*: the model never saw its own predictions as input. The fix is to roll out K steps during training and sum the per-order losses,

$$\mathcal{L}_{\text{pred}} = \sum_{k=1}^{K}\mathcal{L}_k, \qquad z_{t+1}^{(k)} = g_\phi\big(z_{t-v:t}^{(k-1)},\ u_{t-v:t}\big), \quad z_t^{(0)} = f_\theta(x_{t-w:t}), \tag{9}$$

where order k counts predictor calls from a ground-truth representation. On Moving MNIST, downstream Average Precision improves steeply with K and plateaus around $K{=}4$ (Figure 3). In code, K is the `model.steps` (video) or `model.nsteps` (AC-video) config knob, consumed inside `JEPA.unroll`.

Extra world-model regularizers. Action-conditioned training in *randomized* environments needs two more terms (Sobal et al., 2022). A temporal-similarity loss encourages smooth trajectories, and an inverse-dynamics (IDM) loss (LeCun, 2022; Pathak et al., 2017) forces the representation to retain action-relevant information by predicting the action from consecutive states,

$$\mathcal{L}_{\text{sim}} = \sum_t \|z_t - z_{t+1}\|_2^2, \qquad \mathcal{L}_{\text{IDM}} = \sum_t \|a_t - \text{MLP}(z_t, z_{t+1})\|_2^2. \tag{10}$$

The full AC objective combines everything,

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \alpha\mathcal{L}_{\text{var}} + \beta\mathcal{L}_{\text{cov}} + \delta\mathcal{L}_{\text{sim}} + \omega\mathcal{L}_{\text{IDM}}. \tag{11}$$

The ablation in the paper is blunt about how load-bearing each term is: removing the IDM loss collapses Two Rooms planning from 97% to 1% success, because the encoder latches onto spurious background correlations instead of the agent.

2.4 Planning as energy minimization

Once you have a latent dynamics model, planning is optimization. To reach a goal observation x_g, search for an action sequence whose imagined rollout accumulates low energy against the goal representation,

$$E_{\text{plan}}(a_{0:H}; x_0, x_g) = \sum_{t=1}^{H} \|f_\theta(x_g) - \hat{z}_t\|_2^2, \qquad \hat{z}_{t+1} = g_\phi(\hat{z}_{t-v:t}, u_{t-v:t}),\ \hat{z}_0 = f_\theta(x_0). \tag{12}$$

Summing over *all* intermediate steps (rather than only the final state) rewards efficient paths and is robust to compounding prediction error; it beats final-state-only cost by 8 points on Two Rooms. The library solves Equation (12) with two population-based optimizers, MPPI (Williams et al., 2015) (default, soft exponential weighting over elite trajectories) and CEM (hard elite refitting), both in `eb_jepa/planning.py`. You rarely write planning code; you choose a planner config and a cost, and tune the horizon H, the number of samples N, and the number of elites Q.

3 Environment and Compute Setup

Get this working in the first hour. The detailed compute policy for the shared B200 cluster is in Section 7; here we cover installation, the launch workflow, and a smoke test.

3.1 Install

The library uses `uv` for package management. The fastest path:

```
git clone https://github.com/facebookresearch/eb_jepa.git
cd eb_jepa
uv sync                          # create .venv and install deps
source .venv/bin/activate
```

If you prefer conda for system packages:

```
conda create -n eb_jepa python=3.12 -y && conda activate eb_jepa
uv pip install -e . --group dev   # editable install +
    ↪ pytest/black/isort
```

Then point the library at your data and checkpoint directories (add these to `~/.bashrc`):

```
export EBJEPA_DSETS=/path/to/datasets      # where datasets live /
    ↪ download to
export EBJEPA_CKPTS=/path/to/checkpoints   # where runs are written
```

⋆ **Tip** Confirm the install with the test suite before you trust anything: `uv run pytest tests/`. It exercises the loss equivalences, the planning loop, and the JEPA output formats; if it passes, your environment is sane.

3.2 On the hackathon cluster (DALIA, B200)

Repo + venv. Work inside the shared project space `/lustre/work/pdl17890/` — *not* your `$HOME`, whose inode quota blocks `git`. The `uv` virtualenv is *architecture-specific* (login nodes are `x86_64`, compute nodes are `aarch64`), so rather than activating a single `.venv`, source the provided `env.sh`: it selects an arch-specific `UV_PROJECT_ENVIRONMENT`, cache, and `uv` binary, plus `EBJEPA_CKPTS`.

```
cd /lustre/work/pdl17890/<you>/eb_jepa
source env.sh     # -> venvs/eb_jepa_{x86_64|aarch64}; then 'uv pip
    ↪ install ...' as needed
```

SLURM. Submit with `-account=pdl17890 -partition=defq -gres=gpu:1`; exclude the frequently-OOM node with `-exclude=dalianvl03`. The cluster is offline: set `WANDB_DISABLED=true` *and* put `log_wandb: false` in the config — a CLI `-logging.log_wandb=false` is parsed as the string `"false"` (truthy) and still triggers a `wandb` error.

Pre-downloaded datasets (under the shared project space; group-readable):

```
DSETS=/lustre/work/pdl17890/udl806719/datasets
$DSETS/the_well/gray_scott_reaction_diffusion  # The Well / gray_scott
    ↪ (Track 5)
$DSETS/speech_commands_v2                        # Speech Commands v2
    ↪ (Track 3)
$DSETS/modelnet40/modelnet40_ply_hdf5_2048       # ModelNet40
    ↪ (Track 10)
$DSETS/Neuro                                     # EEG corpora
    ↪ (Tracks 1, 2, 6)
```

Point `EBJEPA_DSETS` here (or read them directly); the maze/Two Rooms data is generated on the fly, so it has no path.

3.3 Run something in two minutes

Every example is a `python -m` module that reads a YAML config and accepts dotted overrides on the command line. Start with the image example and a tiny, no-logging configuration to prove the pipeline runs:

```
python -m examples.image_jepa.main \
    --fname examples/image_jepa/cfgs/default.yaml \
    optim.epochs=2 logging.log_wandb=false
```

The three entry points are:

```
python -m examples.image_jepa.main     --fname
    ↪ examples/image_jepa/cfgs/default.yaml
python -m examples.video_jepa.main     --fname
    ↪ examples/video_jepa/cfgs/default.yaml
python -m examples.ac_video_jepa.main  --fname
    ↪ examples/ac_video_jepa/cfgs/train/two_rooms/train.yaml
```

Gotcha. All configs default to `logging.log_wandb: true`. For local smoke tests set `logging.log_wandb=false` or you will hit a `wandb` login prompt. Also note datasets download to `EBJEPA_DSETS`; CIFAR-10 and Moving MNIST fetch automatically on first run (Moving MNIST is ~800 MB and needs internet), while Two Rooms is generated on the fly.

3.4 The SLURM launcher

For anything beyond a smoke test, submit to the cluster with the shared launcher `examples/launch_sbatch.py`. It wraps single jobs, three-seed sweeps, and full hyperparameter grids.

```
# One job (dev mode)
python -m examples.launch_sbatch --example image_jepa \
    --fname examples/image_jepa/cfgs/default.yaml --single

# Three seeds {1, 1000, 10000}, wandb-averaged (recommended default)
python -m examples.launch_sbatch --example image_jepa \
    --fname examples/image_jepa/cfgs/default.yaml --sweep my_experiment

# Full grid from the YAML's sweep.param_grid section
python -m examples.launch_sbatch --example image_jepa \
    --fname examples/image_jepa/cfgs/default.yaml --sweep my_grid \
    --full-sweep --use-wandb-sweep --array-parallelism 8
```

Swap `image_jepa` for `video_jepa` or `ac_video_jepa`. Key flags: `-single` (one dev job), `-sweep NAME` (name your run group), `-full-sweep` (read `sweep.param_grid` from the config), `-use-wandb-sweep` (the wandb sweep UI), and `-array-parallelism N` (cap concurrent jobs, which matters a lot on a shared cluster, see Section 7).

Gotcha. The launcher's default SLURM settings (partition, account, memory) at the top of `examples/launch_sbatch.py` point at the original authors' cluster. On the hackathon cluster use `-account=pdl17890 -partition=defq` (and `-exclude=dalianvl03`), as in Section 3.2. Do not burn an hour debugging a queue rejection; ask. See Section 7.

3.5 Where runs land, and how to compare them

Runs are written under `$EBJEPA_CKPTS/{example}/` with an auto-named experiment folder encoding the key hyperparameters, e.g. `resnet_vicreg_proj_bs256_ep300_std1.0_cov80.0_seed1`. Three-seed sweeps share a wandb run *name* so you can group by name in the UI to get mean and standard error automatically. To compare a before/after change, change *one* thing, keep the same seed set, and read the matched-epoch metric.

⋆ **Tip** The recommended scientific unit is the three-seed sweep, not a single run. JEPA training has real seed variance, and a one-point "improvement" from a single run is usually noise. With B200s you can afford three seeds; budget for it.

4 The Three Examples: Run It, Read It, Tweak It

The library is built around one idea: the training loop, the rollout, and the losses are modality-agnostic and live in the shared `eb_jepa/` package; each example in `examples/` is a thin `main.py` that assembles an encoder, a predictor, and a regularizer, then calls `JEPA.unroll`. Once you have read one example, you have read them all. We go through them in order of increasing complexity. For each: how to run it, the handful of files worth reading, the config knobs that matter, and what to tweak first.

Note. There is no central "registry" of encoders or losses. Components are plain `nn.Modules` wired together by hand inside each example's `main.py`, sometimes behind a small `if` `cfg.model.type == "..."` branch. To add your own, you edit the builder in `main.py` (and occasionally add a class to `eb_jepa/architectures.py` or `eb_jepa/losses.py`). Section 5 is the map.

4.1 Image-JEPA: self-supervised features on CIFAR-10

What it teaches. The purest form of the recipe: no predictor, no time. Two augmented views of an image, pull their representations together, and keep them from collapsing with VICReg or SIGReg. A linear probe trained online reports classification accuracy each epoch (∼91% with SIGReg, ∼90% with VICReg at 300 epochs).

Run it.

```
# ResNet-18 + VICReg (default)
python -m examples.image_jepa.main --fname
    ↪ examples/image_jepa/cfgs/default.yaml
# ResNet-18 + SIGReg
python -m examples.image_jepa.main --fname
    ↪ examples/image_jepa/cfgs/sigreg.yaml
# ViT-S + VICReg
python -m examples.image_jepa.main --fname
    ↪ examples/image_jepa/cfgs/transformers.yaml
```

There is no separate eval step: the probe accuracy `val_acc` is logged every epoch during pretraining.

Read it.

- `examples/image_jepa/dataset.py` returns two augmented views per image (crop, color jitter, grayscale, solarize, flip).
- `examples/image_jepa/main.py`: encoder built at lines `435-468`; loss chosen at `518-522`; the batch loop with the online probe at `275-321`; the epoch loop at `539-605`.
- `eb_jepa/losses.py`: `VICRegLoss` and `BCS` (the SIGReg implementation) take two views and return a loss dict.

Key knobs (`cfgs/default.yaml`).

```
model:  {type: resnet, use_projector: true, proj_hidden_dim: 2048,
    ↪ proj_output_dim: 2048}
loss:   {type: vicreg, std_coeff: 1.0, cov_coeff: 80.0, lmbd: 10.0}  #
    ↪ lmbd used iff type=bcs
optim:  {epochs: 300, lr: 0.3, weight_decay: 1.0e-4}                 #
    ↪ LARS + cosine warmup
data:   {batch_size: 256}
```

Tweak it first.

- Swap the regularizer: `loss.type=bcs loss.lmbd=10`. SIGReg has one coefficient and is more forgiving (Figure 2).
- Swap the backbone: `model.type=vit_s`. Compare convergence and final accuracy at equal epochs.
- Ablate the projector: `model.use_projector=false`. Expect a ∼3 point accuracy drop, a clean demonstration of why the regularizer is computed in a projected space.

Gotcha. Bad regularization weights collapse this model fast. The paper's sweep shows `std=100, cov=100` dropping VICReg to 10% (chance) while `std=1, cov=100` reaches 90%. If `val_acc` is stuck near 10%, you are collapsed, not under-trained. Also: the ViT `patch_size` is currently hardcoded in `main.py`, so the `model.patch_size` YAML key is ignored for ViT.

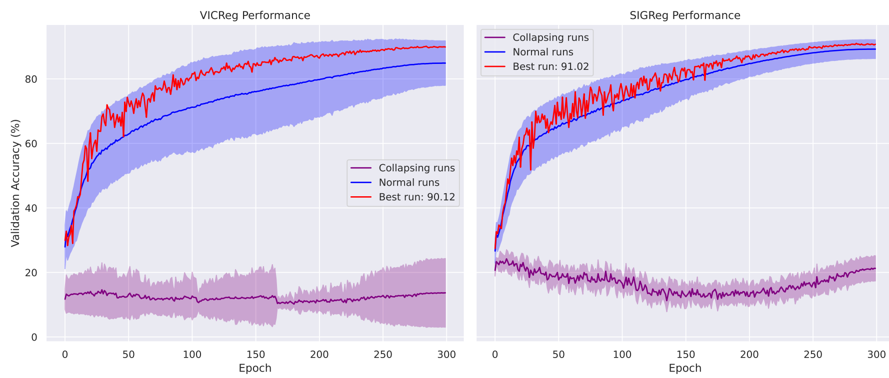

Figure 2: **Hyperparameter sensitivity on CIFAR-10.** SIGReg (right) stays stable across a naive sweep while VICReg (left) reaches a similar peak but needs more careful tuning. Collapsed runs flatline near chance accuracy. Use this as your mental model when a new run will not improve: suspect collapse before suspecting capacity.

4.2 VIDEO-JEPA: TEMPORAL PREDICTION ON MOVING MNIST

What it teaches. The predictor wakes up. Frames are encoded, the past representations are context, and the model predicts the next frame's representation, unrolled K steps to fight exposure bias. A detection head and a pixel decoder (both trained on detached features, purely for evaluation) let you score Average Precision and watch the rollout.

Run it.
```
python -m examples.video_jepa.main --fname
    ↪ examples/video_jepa/cfgs/default.yaml
# fewer rollout steps, larger batch:
python -m examples.video_jepa.main --fname
    ↪ examples/video_jepa/cfgs/default.yaml \
    model.steps=2 data.batch_size=128
```

Read it.

- `eb_jepa/datasets/moving_mnist.py`: `MovingMNISTDet`, two bouncing digits; the dataset, not the example, owns the data.
- `examples/video_jepa/main.py`: model assembled at `128–142` (`ResNet5` encoder, `ResUNet` predictor wrapped by `StateOnlyPredictor`, `VCLoss`, `SquareLossSeq`); training step at `191–204`.
- `eb_jepa/jepa.py`: `unroll` in `parallel` mode, the K-step loop, at `142–157`. This is the heart of temporal training.

Key knobs (`cfgs/default.yaml`).
```
model: {dobs: 1, henc: 32, dstc: 16, hpre: 32, steps: 4}    # steps = K
    ↪ rollout depth
loss:  {std_coeff: 10.0, cov_coeff: 100.0}
optim: {epochs: 50, lr: 1.0e-3}
data:  {batch_size: 64}
```

Tweak it first.

- Sweep the rollout depth `model.steps` $\in \{1, 2, 4, 8\}$ and plot AP versus prediction horizon (Figure 3). This reproduces the paper's central video result in an afternoon.
- Replace the spatial `ResUNet` predictor with the GRU `RNNPredictor` (already in `eb_jepa/architectures.py`) and compare.

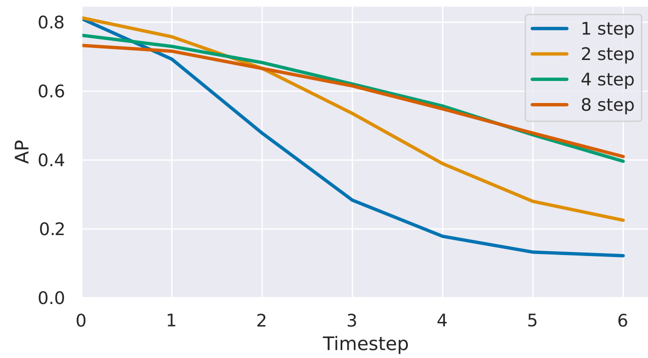

Figure 3: **Multistep rollout helps, and saturates.** Average Precision of the autoregressive prediction versus horizon, for models trained with $K \in \{1, 2, 4, 8\}$ rollout steps. Training with longer rollouts aligns training with autoregressive inference; the Pareto knee is around $K{=}4$. A clean, cheap result to reproduce or extend to a new temporal modality.

4.3 AC-Video-JEPA: a world model you can plan with

What it teaches. The full stack: an action-conditioned latent dynamics model, the two world-model regularizers ($\mathcal{L}_{\mathrm{sim}}$ and the critical $\mathcal{L}_{\mathrm{IDM}}$), and goal-conditioned planning by energy minimization. The agent navigates a "Two Rooms" environment whose wall and door are randomized per trajectory, so the task is non-monotonic (sometimes you must move *away* from the goal to reach it). The best model plans at 97% success with MPPI.

Run it (train, then plan).

```
# Train the world model
python -m examples.ac_video_jepa.main --fname
    ↪ examples/ac_video_jepa/cfgs/train/two_rooms/train.yaml
# Evaluate planning on a trained checkpoint (runs unroll + planning
    ↪ eval)
python -m examples.ac_video_jepa.main \
    --meta.model_folder /path/to/run --meta.eval_only_mode True
```

Planning uses MPPI by default (`cfgs/planning/two_rooms/planning_mppi.yaml`); point `eval.plan_cfg_path` at `cfgs/planning/two_rooms/planning_cem.yaml` for CEM.

Read it.

- `eb_jepa/datasets/two_rooms/`: the environment (`DotWall`), the dataset, and the data-generation pipelines (online / streaming / offline).
- `examples/ac_video_jepa/main.py`: model built at `176-230` (`ImpalaEncoder`, `RNNPredictor`, action encoder `nn.Identity()`, `InverseDynamicsModel`, `VC_IDM_Sim_Regularizer`); training step calling `jepa.unroll(..., unroll_mode="autoregressive")` at `334-343`.
- `eb_jepa/planning.py`: `CEMPlanner` and `MPPIPlanner`, the goal-distance objective `ReprTargetDistMPCObjective`, and the eval entry points `main_eval` / `main_unroll_eval`.

Key knobs.

```
# cfgs/train/two_rooms/train.yaml
model:  {encoder_architecture: impala, dobs: 2, dstc: 32, nsteps: 8}  #
    ↪ nsteps = K
  regularizer: {cov_coeff: 8, std_coeff: 16, sim_coeff_t: 12,
    ↪ idm_coeff: 1}
optim:  {epochs: 12, lr: 0.001}
data:   {batch_size: 384}
# cfgs/planning/two_rooms/planning_mppi.yaml
```

```
planner: {planner_name: mppi, plan_length: 90, n_iters: 20,
    ↪ num_samples: 200, num_elites: 20, temperature: 0.005}
```

Tweak it first.

- Reproduce the headline ablation: set `model.regularizer.idm_coeff=0` and watch planning success collapse toward 1%. Then zero `std_coeff`, `cov_coeff`, `sim_coeff_t` one at a time.
- Compare planners (MPPI vs CEM) and the cost design (`planning_objective.sum_all_diffs` true vs false, i.e. cumulative vs final-state cost). Cumulative is worth ∼8 points.
- Trade planning compute for success: sweep `num_samples` and `plan_length` and report success-versus-wall-clock.

Gotcha. The action dimension `action_dim=2` is hardcoded in several places (`main.py`, `InverseDynamicsModel`, the planners). If you retarget this example to a new control problem, grep for `action_dim` and change all of them together. And remember: in randomized environments the IDM loss is what keeps the encoder honest, do not drop it.

5 Extension Points: The Files You Will Actually Edit

This is the reference you will come back to. Extending the library is not mysterious: there is no plugin system to learn. You add a plain `nn.Module` and wire it into an example's `main.py` builder. Table 2 is the "I want to… → edit this" map; the rest of the section explains the two contracts that matter (the encoder/predictor I/O shape, and the regularizer return signature).

Table 2: **Where to make each kind of change.** "Builder" means the model assembly block near the top of the relevant `examples/*/main.py`.

I want to…	Edit
Add an encoder for a new modality	New `nn.Module` in `eb_jepa/architectures.py`; wire it into the builder (`image_jepa/main.py`∼435, or the hardcoded constructor in `video_jepa/main.py`∼130 / `ac_video_jepa/main.py`∼186).
Add a predictor	New `nn.Module` in `architectures.py` exposing `is_rnn` and `context_length`; wire into the builder.
Add / change a regularizer	New class in `eb_jepa/losses.py`; instantiate in the builder, or add an `elif cfg.loss.type` branch (image).
Add a dataset / modality	New `Dataset` (or `TrajDataset`) under `eb_jepa/datasets/`; for the world-model path, add an `env_name` branch in `eb_jepa/datasets/utils.py:init_data` (it currently rejects anything but `two_rooms`); for the image path, edit the dataset block in `image_jepa/main.py`∼398.
Add a planner or planning cost	New class in `eb_jepa/planning.py` plus an entry in `planner_name_map` / `objective_name_map`.
Change rollout depth, coefficients	Config only: `model.steps`/`model.nsteps`, `std_coeff`, `cov_coeff`, `sim_coeff_t`, `idm_coeff`, `loss.lmbd`.

5.1 The central object: `JEPA.unroll`

Everything funnels through one class, `JEPA` in `eb_jepa/jepa.py`, constructed as `JEPA(encoder, action_encoder, predictor, regularizer, predcost)`. Its `unroll(observations, actions, nsteps, unroll_mode, ...)` method does, in order: encode the observations to `state`; if computing loss, call `regularizer(state, actions)`; encode the actions; then roll the predictor forward and accumulate the prediction cost. Two rollout modes:

- `unroll_mode="parallel"` (used by `video_jepa`): convolutional predictors predict all timesteps at once, refeeding a few ground-truth context frames; the K-step loop is at `jepa.py:142-157`.
- `unroll_mode="autoregressive"` (used by `ac_video_jepa` and any RNN predictor): step-by-step with a sliding context window; loop at `jepa.py:161-190`.

`JEPA` chooses behavior by reading `getattr(predictor, "is_rnn", False)` and `predictor.context_length`. So your predictor mostly needs to declare those two attributes correctly and the rest follows.

5.2 Contract 1: Encoder and predictor tensor shapes

The temporal examples speak in 5D tensors `[B, C, T, H, W]` (batch, channels, time, height, width). The mixin `TemporalBatchMixin` in `eb_jepa/nn_utils.py` folds time into the batch for you, so a 2D encoder can be written as if it only ever sees `[B, C, H, W]`; subclass it and implement `_forward`. Encoders output a representation `[B, D, T, H', W']` (the `ImpalaEncoder` pools to `H'=W'=1`, i.e. one vector per frame). A predictor implements `forward(state, action) -> [B, D, T, ...]` and declares:

```
class MyPredictor(nn.Module):
    is_rnn = False          # True -> autoregressive single-step unroll
    context_length = 2      # how many context frames it consumes
    def forward(self, state, action):
        ...                 # return predicted next-step
    ↪ representation(s)
```

⋆ **Tip** For a new modality with no spatial grid (a 1D time series, a feature vector per step), the cleanest path is the GRU `RNNPredictor` (it sets `is_rnn=True`, `context_length=0`) plus an encoder that outputs `[B, D, T, 1, 1]`. You inherit the autoregressive rollout and planning for free. Reshape your samples to that 5D convention and most of the spatial machinery becomes a no-op.

5.3 Contract 2: The regularizer return signature

There are two regularizer conventions, matching the two settings:

- **Sequence regularizers** (video / world model) accept `(state, actions)` and return a triple `(weighted_loss, unweighted_loss, loss_dict)`. This is what `JEPA.unroll` expects. Models: `VCLoss` (variance+covariance) and `VC_IDM_Sim_Regularizer` (the full world-model regularizer).
- **Two-view SSL losses** (image) accept `(z1, z2)` and return a dict with a `"loss"` key plus per-term entries. Models: `VICRegLoss` and `BCS` (SIGReg). The image training loop logs every extra key in the dict automatically, so new sub-losses show up in logging with no extra work.

Match whichever contract fits your setting and the rest of the loop is untouched. Building blocks you can reuse to compose a new regularizer live in `eb_jepa/losses.py`: `HingeStdLoss` (variance), `CovarianceLoss`, `TemporalSimilarityLoss`, `InverseDynamicsLoss`, and the `BCS` Gaussianity statistic.

5.4 Adding a dataset: the one real dispatch

For image-style two-view data, edit the dataset block in `image_jepa/main.py` (around line 398) to branch on `cfg.data.dataset` and adjust `num_classes` and the normalization constants. For the world-model path, `eb_jepa/datasets/utils.py:init_data` is the single point that maps an `env_name` to data loaders, and it currently raises for any `env_name` other than `two_rooms`. Adding a new environment or trajectory dataset means implementing a `TrajDataset` (which yields `(obs, action, state, ...)` slices via `TrajSlicerDataset`) and adding your branch there. This is the most involved extension, so scope it early.

Gotcha. Per-feature normalization is not optional for new modalities. The variance term in VICReg measures per-dimension standard deviation; if one input channel dwarfs the others (common in EEG, audio, sensor fusion), it will dominate both the encoder and the regularizer. Normalize each channel/feature (z-score or robust scaling) before it enters the encoder.

6 Project Tracks

Below is a menu, not a syllabus. Pick one track, or use one as a springboard for your own idea. Each track names a difficulty, the example it forks, a 24-hour goal, the files you will touch, and a success metric. Difficulty badges: **Warm-up** is reachable by most teams and a safe first project; **Standard** is a solid full-day project; **Ambitious** is high-risk, high-reward, attempt it only if your baseline is already running.

Note. **The reading list.** A curated, per-track bibliography lives in `references/README.md` at the repo root: one folder per paper (`references/paper/<slug>/` with the PDF and a structured `SUMMARY.md`), organized into a **Core** set (EB-JEPA and the JEPA world-model line it builds on) plus three maturity tiers, every row tagged with the modality or track it serves. Find the prior work for your track there, and read its `SUMMARY.md` before the PDF.

What makes a good new-modality target. The headline theme is *continuous, high-dimensional, noisy* data, exactly where predicting in pixel or sample space is wasteful and predicting in a learned latent space shines. A good target has: (i) abundant unlabeled data for pretraining; (ii) a small labeled set for a downstream probe; (iii) temporal or multi-view structure the predictor can exploit; and (iv) enough noise that reconstruction would be a bad idea. EEG (Tracks 1–2) are the worked example of all four.

6.1 Group A: Carry the JEPA recipe to a new modality

Track 1 (Flagship) — Read the Brainwaves: abnormality & epilepsy detection **Warm-up** / **Standard**

Usefulness in neuroscience. This track uses **electroencephalography (EEG)**: non-invasive *neurophysiological recordings* of the brain's electrical activity, picked up by a few dozen electrodes on the scalp. Millions of clinical EEGs are read by hand every year; a model that recovers the clinical label from the raw signal with no hand-engineered features is a step toward automated triage.

The idea. Pretrain a JEPA on raw clinical EEG *without labels*, freeze it, and linear-probe whether the embeddings carry a clinical label that applies to the *whole recording*: normal vs abnormal (TUAB) and epilepsy vs not (TUEP). This mirrors the EEG foundation model LaBraM (Jiang et al., 2024) but swaps its reconstruction objective — a vector-quantized tokenizer that rebuilds the Fourier spectrum — for a pure joint-embedding one: *predict patch representations in latent space*, never the raw signal. That is the point of a JEPA: predicting in representation space lets the target encoder discard the unpredictable parts of low signal-to-noise EEG, rather than forcing a (spectral) reconstruction of it.

Data (preprocessed, ready to probe). Two labeled corpora, one label per recording, already preprocessed on the cluster (each folder ships a `<NAME>_README.md` with the exact task, label layout, and preprocessing):

- **TUAB** (`TUAB_PREPROCESSED`) — binary normal/abnormal, 2,717/276 train/eval recordings, *official patient-disjoint split*.
- **TUEP** (`TUEP_PREPROCESSED`) — patient-level epilepsy vs no-epilepsy, 200 patients (100/100), a *weak* label: the diagnosis is per patient, but epileptiform activity is intermittent, so most windows (often whole recordings) of an epileptic patient carry no visible sign of it. You build a subject-disjoint split and pool per-window scores into a patient decision.

Both: 19-channel 10–20 montage, 200 Hz, 0.1–75 Hz bandpass, 60 Hz notch (TUEP average-referenced). Only *windowing* and *per-channel normalization* are left to you. You pretrain the JEPA on this same EEG while *ignoring* the labels; the labels come back only for the linear probe.

Two framings (start with the first).

- *Temporal* (video-JEPA style, Section 4.2): cut each channel into 1-second patches; encode the context window; predict the *next* 1-second window's representation across all channels. Next-frame prediction with frames = time windows.
- *Augmented-view* (image-JEPA style, Section 4.1): take an epoch (say 10 s) as one "image" and form *two augmented views* of it (two time crops, optionally + masking, amplitude scaling, channel dropout, Gaussian noise); predict one view's representation from the other. The augmentation set is the main design knob.

Files. A new `Dataset` under `eb_jepa/datasets/` that loads the preprocessed EDF (`MNE-Python`) and yields one epoch per sample in eb_jepa's `[B, C, T, 1, 1]` layout — `B` batch, `C` EEG channels, `T` time samples, the trailing `1, 1` the singleton spatial dimensions of the `[B, C, T, H, W]` convention (EEG has no spatial grid). Then a small conv-patch or 1D-conv encoder in `architectures.py`; the GRU `RNNPredictor` for the temporal framing; a linear probe modeled on `examples/image_jepa/eval.py`.

Metric. Balanced Accuracy / AUROC at the **window (epoch) level** — each window inherits its recording's label and is scored individually, as in recent EEG foundation models such as LaBraM and BIOT (TUAB on its official patient-disjoint eval; TUEP on your subject-disjoint split). Context: on other models, a supervised CNN-Transformer reaches ∼0.78 Balanced Accuracy on TUAB, BIOT ∼0.80, and fine-tuned LaBraM-Base ∼0.81; a frozen-JEPA *linear probe* clearing ∼0.78 is a strong result.

Benchmark (optional). Put the frozen-JEPA probe next to a foundation model: run the same linear probe on a pretrained **LaBraM** (Jiang et al., 2024) (fine-tuned) over your split and compare. A frozen JEPA matching a fine-tuned foundation model could be a headline result.

Gotchas. Evaluate only on held-out patients (*patient-disjoint*): EEG carries a per-subject "fingerprint", so an overlapping split decodes *who* not *what*; keep test patients out of both pretraining and the probe fit. The recordings ship continuous — *epoch* them into fixed-length windows (e.g. 10 s) before pretraining or probing. Per-channel normalization is mandatory; use a class-aware metric (eval is mildly imbalanced); score *per window* — each window carries its recording's label, so report metrics over windows, not over collapsed recordings. For TUEP the label is *weak* — most windows of an epileptic patient look normal, so per-window scores are noisy and how you optionally pool them into a patient-level call is the interesting part.

poc: TUAB frozen linear-probe **AUROC 0.877** (patient-disjoint split, near supervised SOTA). TUEP (patient-level epilepsy) is preprocessed and ready as a second, independent task — its own encoder, pretrained and probed within TUEP (*IID*: pretraining and evaluation on the same corpus). *Data:* `$DSETS/Neuro/{TUAB,TUEP}_PREPROCESSED` (each with its `*_README.md`).

Stretch (cross-dataset transfer). Beyond the IID protocol above, the two corpora enable a real generality test: pretrain the JEPA on one (say TUAB), freeze it, then train a *fresh* linear probe on the other's labels (the label spaces differ, so only the encoder transfers). Does a representation learned on one cohort generalize to another? That transfer is exactly what a foundation model claims.

Track 2 — When the Brain Misfires: artifact, epileptiform & seizure detection Standard / Ambitious

Usefulness in neuroscience. Like Track 1 this works on **EEG** (electroencephalography — scalp recordings of the brain's electrical activity), but the clinically meaningful content of a recording is mostly *events* — spikes, seizures, artifacts — sparse in time and localized to a few channels; finding them is the daily work of an epileptologist.

The idea (analogous to Track 1 — pickable on its own). Same recipe: pretrain a JEPA on raw EEG *without labels*, freeze it, and linear-probe whether the embeddings carry the label — a pure joint-embedding objective (*predict patch representations in latent space*, never the raw signal), the same structural alternative to the reconstruction tokenizer of LaBraM (Jiang et al., 2024). Pretrain with either framing from Track 1 (temporal next-window or augmented-view). What changes from Track 1 is *where* the label lives: not one label for the whole recording, but *event-level* labels sitting inside the recording on *specific channels* (see the annotation gotcha below). Three corpora of rising richness:

- **TUAR** (`TUAR_PREPROCESSED`) — per-segment *artifact* detection/typing (eye, muscle, chew, shiver, electrode), 290 recordings; gross and visible, a noise-robust warm-up into the event setting.
- **TUEV** (`TUEV_PREPROCESSED`) — 6-class event classification (spike/sharp-wave, two periodic-discharge types GPED/PLED, eye-movement, artifact, background), 359/159 train/eval, imbalanced.
- **TUSZ** (`TUSZ_PREPROCESSED`) — *seizures*, the richest: each recording ships *two* annotations, a binary whole-brain `seiz`/`bckg` timeline (detection) and a per-channel *seizure type* (8 classes, typing); 7,515 recordings, official train/dev/eval, $\sim$13% seizure-bearing.

The annotation gotcha (read the README). TUEV/TUAR/TUSZ events are time-stamped on a *bipolar* montage: each annotation gives a channel that names a *pair* of electrodes (not a row of the signal), a start and end time, and a label — so turning those annotations into training samples is itself part of the task. Each folder's `<NAME>_README.md` spells this out.

Data (preprocessed). All three at 200 Hz, 0.1–75 Hz bandpass, 60 Hz notch (TUSZ average-referenced; 19–23 channels depending on the bipolar montage the labels need). The recordings ship continuous — you must first *epoch* them into fixed-length windows; windowing and per-channel normalization are left to you.

Gotcha (patient-disjoint). Evaluate only on held-out patients — EEG carries a per-subject "fingerprint", so an overlapping split decodes *who* not *what*; keep test patients out of pretraining and the probe fit. TUEV/TUSZ ship official splits; build your own (patient-disjoint) for TUAR.

Files. A `Dataset` that loads the preprocessed EDF (`MNE-Python`) and yields one epoch per sample in eb_jepa's `[B, C, T, 1, 1]` layout (`B` batch, `C` EEG channels, `T` time samples; the trailing `1, 1` are the singleton spatial dims of the `[B, C, T, H, W]` convention). Then a conv-patch / 1D-conv encoder in `architectures.py`; the GRU `RNNPredictor` for the temporal framing; a linear probe modeled on `examples/image_jepa/eval.py` — *plus* per-event window extraction from the `.csv` / `.csv_bi` annotations (the event-specific part).

Metric. Per-window or per-event balanced F1 / AUROC; for TUSZ do *detection* (binary, every window labeled, whole-brain — tidiest) before *typing* (multi-class, per-channel, imbalanced). Design the metric so a `bckg`-only predictor does not look good (events/seizures are rare).

Benchmark (optional). Compare your event-level probe against a fine-tuned **LaBraM** (Jiang et al., 2024) foundation model on the same windows and splits.

poc: on the TUEV 6-class event task, frozen-JEPA $\sim$0.68 AUROC — vs the easy TUAB 0.877 of Track 1, this is where the recipe strains. TUAR and TUSZ are preprocessed and ready as further *independent* tasks — each its own encoder, pretrained and probed within that corpus (*IID*). *Data:* `$DSETS/Neuro/{TUAR,TUEV,TUSZ}_PREPROCESSED` (each with its `*_README.md`).

Stretch (cross-dataset transfer). Beyond the IID setting, the three event corpora are a natural transfer test: pretrain the JEPA on one, freeze it, then train a *fresh* probe on another's events (the label spaces differ, so only the encoder transfers). Does an encoder trained on one kind of event generalize to another? That generality is exactly what a foundation model claims.

Track 3 — Audio / speech representation JEPA

Standard

Goal. Self-supervised audio features, probed on keyword spotting (Speech Commands) or environmental sound (ESC-50/UrbanSound). Treat log-mel spectrograms as single-channel "images" for the image-JEPA masked/augmented setting, or treat mel frames as a temporal sequence for the video-JEPA setting. **Files:** a spectrogram dataset; reuse the ResNet or ViT encoder. **Metric:** linear-probe accuracy on the downstream classification set. **Why JEPA:** audio is continuous and noisy; latent prediction avoids modeling phase and background noise.

poc: Speech Commands (35-class, chance 2.86%): linear-probe **72.6%** with log-mel spectrograms vs **33.2%** with the raw 1D waveform; the augmentation (VICReg) variant beat a masked-predictive one. *Data:* `/lustre/work/pdl17890/udl806719/datasets/speech_commands_v2` | Speech Commands v0.02 (Google; not on HF).

Track 4 — Wearable biosignals: ECG / IMU / PPG Warm-up / Standard

Goal. JEPA on accelerometer/gyroscope or ECG streams; probe on Human Activity Recognition (UCI-HAR, PAMAP2) or arrhythmia (PhysioNet). These are low-spatial multivariate time series, the ideal fit for the GRU predictor and `[B, C, T, 1, 1]` convention. **Files:** a windowed time-series dataset; 1D-conv encoder; `RNNPredictor`; linear probe. **Metric:** probe accuracy / F1. The lowest-friction new-modality track, good for a first-time team.

Track 5 — Latent dynamics of physical fields (The Well) Standard / Ambitious

Goal. The Well (Ohana et al., 2024) is a 15 TB collection of 16 physics-simulation datasets (fluid dynamics, magneto-hydrodynamics, astrophysics, acoustics, active matter) stored as spatiotemporal fields, the continuous, high-dimensional modality JEPAs were made for, with a ready-made PyTorch loader. Train a video-JEPA-style model to predict the next field state in latent space, then answer the open question: does latent prediction give more stable long-horizon rollouts than the field-space neural-operator surrogates (FNO, U-Net) that The Well ships as baselines? **Files:** wrap `the_well.data.WellDataset` to emit `[B, C, T, H, W]` clips; reuse the `ResUNet` spatial predictor and `VCLoss`; add a small decoder from latent back to the field for scoring. **Data.** Start at the small end of the 6.9 GB–5.1 TB range, a 2D set such as `active_matter` or `gray_scott_reaction_diffusion`; do not pull the multi-TB sets. **Metric:** VRMSE of the decoded autoregressive rollout vs the FNO/U-Net baseline at several horizons. A clean JEPA-versus-surrogate comparison on real physics (for *observational* rather than simulated fields, ERA5/WeatherBench is a harder variant).

poc: on `gray_scott_reaction_diffusion`, the latent rollout beats a persistence baseline **4.5**$\times$ at horizon 6 and frozen latents separate the 6 regimes at **0.92** accuracy (chance 0.167). But in decoded field-space VRMSE the FNO/U-Net surrogates win at *every* horizon $\leq$10 — a latent$\to$field decoder floor ($\approx$0.19 VRMSE) caps the JEPA. Answer to the open question: here the gain is representational, not pixel-accurate rollout. *Data:* `/lustre/work/pdl17890/udl806719/datasets/the_well/gray_scott_reaction_diffusion` | HF polymathic-ai/gray_scott_reaction_diffusion.

Action-conditioned prediction: planning and forecasting (Tracks 6–9). The action-conditioned tracks below share one mapping onto the codebase. The AC-video-JEPA (Section 4.3) is a latent dynamics model: given a state and an *action*, it predicts the next state's representation. The "action" is whatever exogenous or control signal drives the system — a task *cue* for EEG motor imagery (Track 6), or actuator commands, manipulated set-points, the operating regime, known future covariates, or order-flow events for the time-series tracks (7–9) — and "predict the future" is forecasting the next state in latent space. Anomaly detection then falls out for free: a fault or an attack is a *violation of expectation*, a spike in prediction energy against the *realised* future (rather than against a planning goal) that the action-conditioned dynamics cannot explain—the same signal the intuitive-physics track uses on impossible videos. Concretely: fork the AC-video-JEPA, feed it `(obs, action)` slices through a new `TrajDataset` (Section 5), drop the planner, and read off either the rollout forecast or the per-step energy. Two gotchas apply to all four: the hardcoded `action_dim` (Section 4.3) and mandatory per-channel normalization (Section 5). With no control channel, the same recipe degenerates to the video-JEPA forecasting setting (Section 4.2).

Track 6 — Mind in Motion: action-conditioned motor-imagery prediction (BCI IV-2a)

Standard / Ambitious

Usefulness in neuroscience. Decoding *intended* movement from EEG (electroencephalography — scalp recordings of the brain's electrical activity) is the core of brain–computer interfaces that let paralysed patients drive a cursor or a prosthetic.

The idea (three parts, easy→hard — attempt all three). BCI IV-2a is cue-based motor imagery: on each trial the subject imagines moving the left hand, right hand, feet, or tongue. Every part starts from the *same* JEPA encoder pretrained self-supervised on these recordings (no labels); unlike the other tracks, here you are expected to work through all three:

1. *Within-subject probe (warm-up).* Freeze the encoder and linear-probe the 4 classes per subject, fitting on session `T` and testing on session `E` (same person, different day) — the original competition protocol.
2. *Cross-subject probe (hard).* The same probe, but *leave-subjects-out*: fit on some subjects, test on held-out ones. Motor-imagery patterns are very person-specific, so this is the honest subject-disjoint test.
3. *Action-conditioned prediction (stretch).* The cue is an externally-imposed *action*, so this is the one EEG corpus that maps onto the AC-video-JEPA (Section 4.3): encode the pre-cue baseline, condition the predictor on the cue, predict the post-cue imagery representation, and decode the cue by which action best reproduces the realised future (the inverse-dynamics head, or energy over the 4 actions). Encoder and predictor are trained *jointly* with var/cov+IDM anti-collapse and no EMA target.

Data (preprocessed). `BCICIV_2a_PREPROCESSED` — 9 subjects $\times$ 2 sessions, 288 balanced trials/session, 22 EEG channels at 250 Hz, already 0.5–100 Hz bandpass + 50 Hz notch (European data; the 3 EOG channels are dropped). A per-trial cue+label `.csv` is provided for *both* sessions; `BCICIV_2a_README.md` gives the cue timing (t=2 s), the imagery window $[2, 6]$ s, and the labels.

Files. For the warm-up, a `Dataset` that loads the preprocessed EDF (`MNE-Python`) and yields one epoch per sample in eb_jepa's `[B, C, T, 1, 1]` layout (`B` batch, `C` EEG channels, `T` time samples; the trailing `1, 1` are the singleton spatial dims of the `[B, C, T, H, W]` convention), a conv-patch / 1D-conv encoder in `architectures.py`, and a linear probe modeled on `examples/image_jepa/eval.py`. For the stretch, a `TrajDataset` (Section 5) yielding `(pre-cue, cue, post-cue)` slices into the AC-video-JEPA, the GRU `RNNPredictor`, and the IDM head as the cue decoder. Set the hardcoded `action_dim` to the 4-way cue.

Metric. 4-class accuracy / Cohen's κ on held-out sessions (part 1) and held-out subjects (part 2). For the action-conditioned part 3: does conditioning on the *correct* cue predict the realised future better than a wrong one, and what is the IDM cue-decoding accuracy? Judge it on whether conditioning changes the prediction at all, not on raw accuracy.

Benchmark (optional). For the probe parts, compare against a pretrained **LaBraM** (Jiang et al., 2024) foundation model on the same within-/cross-subject splits.

Gotchas. The EDF is continuous — epoch each trial to its cue-locked imagery window $[2, 6]$ s before probing. Tiny (∼288 trials/subject); cross-subject is genuinely hard (person-specific sensorimotor rhythms); EOG must never enter the classifier (already dropped); 50 Hz notch, not 60. Parts 1–2 are the safe deliverable; part 3 is the ambitious one. *Data:* `$DSETS/Neuro/BCICIV_2a_PREPROCESSED` (with `BCICIV_2a_README.md`).

Track 7 — Scientific multivariate time-series forecasting

Standard / Ambitious

Goal. Pretrain a JEPA to predict the next window of a noisy scientific multivariate series in latent space, then probe it on long-horizon forecasting or regime classification. The research question: does anti-collapse latent prediction learn more transferable, less overfit features than direct forecasting on non-stationary signals?

Data. Use the *de facto* long-horizon forecasting (LTSF) suite, shipped with fixed train/val/test splits and ∼30 baselines in the Time-Series-Library (`thuml/Time-Series-Library`) and introduced by Informer (Zhou et al., 2021) and Autoformer (Wu et al., 2021): **Weather** (21 meteorological channels, 10-min), **ETT** (transformer oil temperature + 6 power-load channels, hourly / 15-min), **Electricity** (321 clients, hourly), **Traffic** (862 road sensors, hourly), and **ILI** (weekly influenza-like-illness counts). For breadth across scientific domains (solar, temperature, hospital, nature), the Monash archive (Godahewa et al., 2021) offers $\sim$25 datasets in a single `.tsf` format (`Monash-University/monash_tsf` on Hugging Face). For spatiotemporal *fields* rather than vector series, see Track 5 (The Well) and ERA5/WeatherBench.

Action-conditioned angle. **ETT** is the cleanest fit: condition the predictor on the six power-load channels (the "action") and forecast the oil temperature—a genuinely controlled physical system (thermal response to load). More generally, any dataset with *known-future* exogenous covariates (calendar, weather driving electricity / traffic) lets `q_omega` encode the covariate as the action; with none, drop to the video-JEPA forecasting setting.

Files. A windowed multivariate dataset wrapping the TSLib loaders (reuse their splits verbatim for comparability); a 1D-conv or patch encoder in `architectures.py`; the GRU `RNNPredictor` and the `[B, C, T, 1, 1]` convention.

Metric. Forecasting MSE / MAE at the standard horizons $\{96, 192, 336, 720\}$ (ILI: $\{24, 36, 48, 60\}$) against the published TSLib numbers (DLinear, PatchTST, iTransformer, TimesNet), or a frozen-feature probe vs. a supervised baseline on the same split. Beating a strong linear baseline (DLinear) is the bar worth clearing.

poc. Tested on **ETTh1** (`ailuntz/ETT-small`), TSLib 12/4/4 split, input $L = 336$, normalized MSE/MAE on test. A predictive JEPA (1D conv encoder + `RNNPredictor` + EMA target + VICReg anti-collapse) was pretrained SSL on the train-window inputs, then a frozen-feature ridge head forecast horizons $\{96, 192, 336, 720\}$, against a random-encoder floor, an end-to-end supervised head, and an NLinear/DLinear baseline. *The linear baseline dominates by* $\sim$*5*$\times$ (MSE **0.37 / 0.40 / 0.43 / 0.43**, matching the published DLinear numbers — confirming the split/normalization are correct), while frozen-JEPA $\approx$ random $\approx$ supervised (MSE $\approx$1.2–2.9). Crucially the *whole global-pooled conv-encoder family* fails, not JEPA specifically: the supervised version of the same backbone also loses to the linear map, reproducing the Zeng et al. (DLinear) lesson that a pooled representation discards the level / last-value information ETT rewards; latent prediction adds no transfer under this probe. *Caveat:* the probe forecasts from a single time-pooled vector — a fairer JEPA-for-LTSF eval would keep a per-patch / per-timestep representation (PatchTST-style) or fine-tune end-to-end, so this is evidence against *a global-pooled conv-JEPA representation* here, not against JEPA forecasting in general (n=1 seed, ETTh1 only). *Data:* `/lustre/work/pdl17890/udl806719/datasets/LTSF/ETT-small` | HF ailuntz/ETT-small.

Track 8 — Financial time series: limit order books & forecasting Standard / Ambitious

Goal. Self-supervised features on high-frequency market data, probed on mid-price-movement prediction. The question: can latent prediction extract a usable signal from order-book dynamics that survives the brutal noise floor of financial data?

Data. FI-2010 (Ntakaris et al., 2018) is the standard public limit-order-book (LOB) benchmark—often called the "MNIST of high-frequency trading"—with $\sim$4M samples from five Nasdaq Nordic stocks over ten days, the top ten LOB levels (40 features: bid/ask price and volume), and 3-class mid-price-direction labels (up / stationary / down) at horizons $k \in \{10, 20, 30, 50, 100\}$ ticks; the standard protocol trains on the first seven days and tests on the last three. For lower-frequency forecasting, the **Exchange-Rate** set (eight countries' daily rates) (Lai et al., 2018) sits in the same LTSF suite as Track 7, and the Monash archive (Godahewa et al., 2021) adds further financial series.

Action-conditioned angle. The order book evolves under a stream of events—placements, executions, cancellations. Treat that order flow (or order-flow imbalance) as the "action" driving the book: condition the predictor on the incoming event and forecast the next LOB state's representation, then read the mid-price direction off the predicted latent.

Files. A LOB dataset (FI-2010 ships normalized; public PyTorch loaders exist, e.g. the DeepLOB repositories); a temporal / 1D encoder; `RNNPredictor`.

Metric. 3-class mid-price F1 / accuracy at the standard horizons (frozen probe vs. supervised), or forecasting MSE / MAE on Exchange-Rate.

Gotcha. Markets are near-efficient and adversarial; mid-price direction is genuinely hard and reported scores swing wildly with the label threshold and horizon. A clean controlled comparison (JEPA features vs. supervised, with the collapse you fought) is the win here—*not* a number that looks like alpha. Treat any "profitable" result with deep suspicion and check for look-ahead leakage first.

poc. Tested on thesven/fintime-decoder-dataset (697 instruments, 2010–2025; windows of 87 engineered features $\times$ 64 daily steps). A predictive JEPA (1D conv encoder + `RNNPredictor` + EMA target + VICReg anti-collapse) was pretrained SSL, frozen, and probed against a random-encoder floor and an end-to-end supervised baseline. *Negative result, exactly as the gotcha warns:* on the genuinely hard targets (next-step direction / return) the frozen JEPA *does not beat the random encoder* (direction AUROC **0.58** vs random 0.58 vs supervised 0.69), and this still holds on raw *OHLCV-only* inputs (6 channels); returns are $\approx$chance out-of-distribution for every method. The only non-trivial self-supervised signal comes from *VICReg, not latent prediction* (volatility-regime acc **0.94** vs random 0.63). *Takeaway:* daily returns here are too close to efficient-market noise and the engineered inputs are already informative, so this dataset cannot separate latent-prediction from direct forecasting — for a fair test of the track's hypothesis prefer a structured series with learnable dynamics (the LTSF / Exchange-Rate suites of Track 7, or FI-2010 LOB above). *Data:* `/lustre/work/pdl17890/udl806719/datasets/Finance/fintime_prep` | HF thesven/fintime-decoder-dataset.

Track 9 — Action-conditioned anomaly prediction & predictive maintenance Standard / Ambitious

The idea. A JEPA trained only on *healthy* operation learns the action-conditioned dynamics of a system; at test time a fault, attack, or degradation shows up as a spike in prediction energy—the model expected one future and reality delivered another. This is forecasting-based anomaly detection, and it is exactly the violation-of-expectation signal the intuitive-physics track uses, now on real industrial data with a genuine control channel.

Data (three flavours, each with an action channel).

- *Cyber-physical attacks.* **SWaT** and **WADI** (Goh et al., 2016; Ahmed et al., 2017) are water treatment / distribution testbeds (iTrust, SUTD; request access) with both sensors *and actuators*: SWaT has 51 channels over 7 normal + 4 attack days, WADI 123 channels over 14 + 2. Condition on the actuator commands (pump / valve states); the attacks are the anomalies.
- *Process faults.* The **Tennessee Eastman Process** (Downs & Vogel, 1993) is the canonical closed-loop chemical-plant benchmark: 41 measured + *11 manipulated* variables, 20 fault types, with a large multi-run simulation set (Rieth et al., 2017) (also packaged by `AIRI-Institute/fddbenchmark`). The manipulated variables are the control actions, and the hard part is telling a fault from a normal closed-loop control response.
- *Run-to-failure / RUL.* **NASA C-MAPSS** (Saxena et al., 2008) is *the* prognostics benchmark: turbofan run-to-failure trajectories, 21 sensors + *3 operational settings* (altitude, Mach, throttle), four subsets FD001–FD004 of rising difficulty. Condition on the operational settings, track the drift of the latent, and regress remaining useful life.

Methodology caveat. The popular multivariate sets are flawed: Wu & Keogh (Wu & Keogh, 2023) show the NASA telemetry sets (SMAP / MSL) contain trivial, mislabeled, end-loaded anomalies, and even SWaT has many constant channels and anomalies detectable from a single feature. For a clean, sound *univariate* evaluation use their **UCR Anomaly Archive** (250 curated signals)—it has no action channel, so use it to validate the detector and SWaT / TEP / C-MAPSS to exercise the action-conditioned story. Report event / range-based F1 and AUROC (VUS-PR), not just point-adjusted F1, which inflates scores.

Files. A new `TrajDataset` yielding `(obs, action)` slices (Section 5) plus an `env_name` branch in `datasets/utils.py:init_data`; a 1D-conv encoder; `RNNPredictor`; the AC-video-JEPA dynamics model (Section 4.3) with the planner removed and a prediction-energy anomaly score. Watch the hardcoded `action_dim` (SWaT / WADI: dozens of actuators; TEP: 11; C-MAPSS: 3) and normalize every channel.

Metric. SWaT / WADI / TEP: event-based F1 and AUROC of the energy score, healthy-only training. C-MAPSS: RUL RMSE and the NASA asymmetric score (which penalises late predictions more) on FD001 first, then the harder FD004.

Track 10 — 3D point clouds / LiDAR Ambitious

Goal. View-invariant JEPA on point clouds (ModelNet/ShapeNet): two augmented samplings/rotations of an object are the two views (image-JEPA setting), probed on shape classification. **Files:** a point-cloud dataset with geometric augmentations; a PointNet-style encoder in `architectures.py`; reuse `VICRegLoss`. **Metric:** linear-probe classification accuracy. Tests whether the recipe transfers to an unordered, irregular modality.

poc: ModelNet40 (40-class, chance 2.5%): linear-probe **80.9%** with aligned views, **64.7%** under azimuth rotations, **35.3%** under full SO(3) — accuracy drops monotonically as the demanded rotation invariance grows. *Data:* `/lustre/work/pdl17890/udl806719/datasets/modelnet40/modelnet40_ply_hdf5_2048` | HF Msun/modelnet40.

6.2 Group B: Push the vision and robotics examples

Track 11 — Learned cost / value for planning Standard / Ambitious

Goal. Distance-in-latent-space is a crude planning cost. Replace it with a learned cost or value function (TD-MPC style (Hansen et al., 2024; 2022)) trained on the world model's own rollouts, so the agent optimizes a quantity that correlates with task success rather than raw representation distance. **Files:** a new objective class in `eb_jepa/planning.py` registered in `objective_name_map`; a small value head trained alongside the JEPA. **Metric:** Two Rooms success and planning compute vs the distance-cost baseline, especially on the hardest non-monotonic episodes. Attacks a limitation the paper explicitly calls out as open.

poc: on a frozen maze world model, the learned value reaches **37.5%** planning success vs **6.25%** for the distance cost ($\sim 6\times$) under matched A* subgoals; greedy global planning is 0% for both costs. *Data:* maze generated online (no dataset path / HF).

Track 12 — Hierarchical / multi-timescale world model Ambitious

Goal. JEPAs here predict at a single temporal resolution; intelligent planning wants several. Add a coarse predictor that models long-horizon abstractions on top of the fine per-step predictor, and plan with the coarse level to cut the search horizon. The paper names its modular encoder/predictor/regularizer split as "a natural starting point" for exactly this, and no example ships it yet. **Files:** a second predictor and a two-level unroll in a forked `ac_video_jepa/main.py`; a coarse-level temporal-similarity term. **Metric:** long-horizon planning success and wall-clock vs the flat baseline. The deepest open robotics direction in the guide.

poc: on 21×21 mazes, a two-level hierarchy solves them **A*-free at 65.6% success / SPL 0.62** (matched-budget controls: 0% for greedy and 0% for a random-walk baseline). Co-training the two levels did *not* beat the frozen single-level world model. *Data:* maze generated online (no dataset path / HF).

Track 13 — Stress-test the recipe under factors of variation Ambitious

Goal. `eb_jepa`'s Two Rooms is deliberately tiny. The open question is whether the *minimal* recipe (VC + sim + IDM, distance-cost MPPI) survives realistic perturbations, and which term breaks first. Train an AC-video-JEPA-style model and evaluate planning as you dial up controllable visual, geometric, and physical factors of variation, using the environment suites in `stable-worldmodel` (Maes et al., 2026) or the diverse planning benchmarks in `jepa-wms` (Terver et al., 2026b) rather than hand-building a new environment. **Files:** adapt one of those suites' envs to the `eb_jepa` JEPA/planning stack (encoder + `JEPA.unroll` + a planner). **Metric:** a success-vs-perturbation curve against the Two Rooms baseline, and which regularizer term must be strengthened as the world gets harder. Ambitious but high-signal: it tells you where the toy recipe actually ends.

Track 14 — Does intuitive physics emerge? Standard

Goal. Recent work shows physical intuition emerges from self-supervised video pretraining (Garrido et al., 2025). Test it in miniature on the video-JEPA: feed it physically plausible vs impossible Moving MNIST sequences (a digit teleporting, passing through a wall, or reversing instantly) and ask whether prediction energy in latent space spikes on the impossible ones, a violation-of-expectation signal. **Files:** a probe that constructs paired plausible/impossible clips and compares `predcost` energy; light additions to the video eval. **Metric:** the energy gap (impossible $>$ plausible) and how it grows over training. Exploratory, cheap, and genuinely open at this scale.

⋆ **Tip** Whatever you pick, the deliverable that wins is a *controlled comparison*: a baseline and one change, three seeds each, one figure. "We ported JEPA to EEG and the linear probe hits 0.79 Balanced Accuracy, and here is the collapse we hit at zero covariance weight and how we fixed it" beats a pile of half-finished ideas.

7 Compute, Logistics, and Judging

7.1 The compute budget

We share **72 NVIDIA B200 GPUs** across **25 teams** for 24 hours. That is about **2.9 GPUs per team** on average. The good news: every example in this library is designed for *single-GPU* training and finishes in a few hours, so a team's natural unit of work is "a few single-GPU jobs in parallel," not one giant distributed run. A three-seed sweep is three single-GPU jobs; a small hyperparameter grid is a handful more.

Note. The numbers and the SLURM details below are the organizers' *proposed* policy. The exact `-partition`, `-account`, per-GPU memory, and any fair-share quotas will be confirmed at kickoff and pinned in the event channel. If a launch is rejected by the scheduler, it is almost certainly a partition/account mismatch, ask in the channel rather than debugging it alone.

Fair-share rules of thumb.

- Prefer many *single-GPU* jobs over multi-GPU jobs. The examples do not need more than one GPU, and single-GPU jobs schedule faster and waste less.
- Cap your concurrency with `-array-parallelism`. A sane ceiling is **3 concurrent GPUs per team** during the day; if the queue is empty overnight, use more, but yield when teams are waiting.
- Kill dead runs. A collapsed run (flat variance, chance accuracy) will not recover; cancel it and free the GPU rather than letting it burn 12 hours.
- Smoke-test on `-single` with `optim.epochs` tiny before launching a sweep. A typo that crashes at epoch 0 across 12 array tasks wastes everyone's slots.

Disk and checkpoints. Runs write to `$EBJEPA_CKPTS`. Checkpoints add up fast across a sweep; keep only the best and the latest, and clean up smoke tests. Datasets live in `$EBJEPA_DSETS`; download a shared copy once per node rather than per team where possible (the organizers will pre-stage CIFAR-10, Moving MNIST, and any agreed new-modality datasets).

7.2 Notes on B200 vs the default H100 configs

The shipped configs are tuned for H100s. B200s have substantially more memory and strong `bfloat16` throughput, so the defaults will run but leave performance on the table. Practical adjustments:

- **Increase batch size.** The image example uses `batch_size=256` and the world model `384`; you can likely push these up on a B200. Scale the learning rate accordingly and re-check for collapse.
- **Keep `bfloat16`.** The configs already set `use_amp: true` with `bfloat16` (image, world model) or `float16` (video). Prefer `bfloat16` on B200; if you see NaNs under `float16`, switch the video config to `bfloat16`.
- **Try `torch.compile`.** The world-model example already supports it; compilation pays off most on the longer temporal rollouts.
- **Do not over-scale.** A bigger batch is not free accuracy; for SSL it changes the regularization dynamics. Treat any batch-size change as an ablation with its own before/after, not a default win.

> ⋆ **Tip** The bottleneck for these small models is often the data loader, not the GPU. If GPU utilization is low, raise `data.num_workers` and enable `persistent_workers`/`pin_mem` before reaching for a bigger model. For a new modality, a slow EDF/NetCDF reader can starve a B200 completely; pre-process to a fast on-disk format (memmap, `.npy`, webdataset) once, up front.

7.3 Deliverables

By the 24-hour mark each team submits:

1. **A repository**, submitted via **GitHub Classroom** (one repo per group): your fork/branch with all changes, runnable from a single documented command, plus the config(s) you used and a short report (hypothesis, setup, result with at least one figure or table, and one paragraph on what you learned about JEPAs — especially any collapse you fought).
2. **A 10-minute presentation** to the room (see Section 7.4): your hypothesis, the controlled comparison, and one figure or live/recorded artifact (a probe-accuracy curve, a planning rollout GIF, a latent prediction).

7.4 The presentation

To be done.

7.5 How projects are judged

Table 3 is the rubric. Note what is *not* on it: raw leaderboard position. A negative result, cleanly shown ("latent prediction did not beat the supervised baseline on this modality, and here is the evidence and our hypothesis"), scores well. Confident hand-waving does not.

7.6 Getting unstuck

Read the gotcha boxes in Section 4 first; most early problems are there. For collapse, re-read Section 2 and watch the per-term losses. For anything cluster-related, use the event channel. Mentors will run office hours on a posted schedule. And remember the meta-advice: lock scope early, get a pipeline running in hour five, and stop building in time to measure. Have a great 24 hours.

Table 3: **Judging rubric.** Equal weight on understanding and execution; creativity and clarity break ties.

Criterion	What we look for
JEPA understanding	Can the team explain encoder/predictor/regularizer, why collapse happens, and how their design avoids it?
Scientific rigor	A clear hypothesis and a controlled before/after comparison; seeds and error bars where feasible.
Result	A real, interpretable outcome (positive or negative) with a figure/number, not just "it ran."
Creativity	A modality or idea that genuinely stresses the recipe, or an elegant solution to a collapse/scaling problem.
Clarity	A repo and presentation a non-expert teammate could follow.

References

Chuadhry Mujeeb Ahmed, Venkata Reddy Palleti, and Aditya P. Mathur. WADI: a water distribution testbed for research in the design of secure cyber physical systems. In *International Workshop on Cyber-Physical Systems for Smart Water Networks (CySWATER)*, pp. 25–28, 2017.

Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *CVPR*, 2023.

Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba, Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-jepa 2: Self-supervised video models enable understanding, prediction and planning, 2025.

Randall Balestriero and Yann LeCun. Contrastive and non-contrastive self-supervised learning recover global and local spectral embedding methods. In *NeurIPS*, NIPS '22, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088.

Randall Balestriero and Yann Lecun. How learning by reconstruction produces uninformative features for perception. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (eds.), *ICML*, volume 235 of *Proceedings of Machine Learning Research*, pp. 2566–2585. PMLR, 21–27 Jul 2024. URL https://proceedings.mlr.press/v235/balestriero24b.html.

Randall Balestriero and Yann LeCun. Lejepa: Provable and scalable self-supervised learning without the heuristics, 2025. URL https://arxiv.org/abs/2511.08544.

Randall Balestriero, Hugues Van Assel, Sami BuGhanem, and Lucas Maes. stable-pretraining-v1: Foundation model research made simple, 2025. URL https://arxiv.org/abs/2511.19484.

Amir Bar, Gaoyue Zhou, Danny Tran, Trevor Darrell, and Yann LeCun. Navigation world models. In *CVPR*, pp. 15791–15801, June 2025.

Adrien Bardes, Jean Ponce, and Yann LeCun. Vicreg: Variance-invariance-covariance regularization for self-supervised learning. In *ICLR*, 2022.

Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mido Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856.

Andreas Blattmann, Tim Dockhorn, Sumith Kulal, Daniel Mendelevitch, Maciej Kilian, Dominik Lorenz, Yam Levi, Zion English, Vikram Voleti, Adam Letts, Varun Jampani, and Robin Rombach. Stable video diffusion: Scaling latent video diffusion models to large datasets, 2023. URL https://arxiv.org/abs/2311.15127.

Tim Brooks, Bill Peebles, Connor Holmes, Will DePue, Yufei Guo, Li Jing, David Schnurr, Joe Taylor, Troy Luhman, Eric Luhman, et al. Video generation models as world simulators, 2024. URL https://openai.com/research/video-generation-modelsas-world-simulators.

Jake Bruce, Michael D Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, et al. Genie: Generative interactive environments. In *ICML*, 2024.

Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *ICML*, 2020.

Xinlei Chen and Kaiming He. Exploring simple siamese representation learning. In *CVPR*, 2021.

Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, pp. 02783649241273668, 2023.

Kenneth James Williams Craik. *The nature of explanation*, volume 445. CUP Archive, 1967.

Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.

J. J. Downs and E. F. Vogel. A plant-wide industrial process control problem. *Computers & Chemical Engineering*, 17(3):245–255, 1993.

Lasse Espeholt, Hubert Soyer, Remi Munos, Karen Simonyan, Vlad Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, Shane Legg, and Koray Kavukcuoglu. IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In Jennifer Dy and Andreas Krause (eds.), *ICML*, volume 80 of *Proceedings of Machine Learning Research*, pp. 1407–1416. PMLR, 10–15 Jul 2018. URL https://proceedings.mlr.press/v80/espeholt18a.html.

Quentin Garrido, Mahmoud Assran, Nicolas Ballas, Adrien Bardes, Laurent Najman, and Yann LeCun. Learning and leveraging world models in visual representation learning, 2024.

Quentin Garrido, Nicolas Ballas, Mahmoud Assran, Adrien Bardes, Laurent Najman, Michael Rabbat, Emmanuel Dupoux, and Yann LeCun. Intuitive physics understanding emerges from self-supervised pretraining on natural videos, 2025. URL https://arxiv.org/abs/2502.11831.

Rakshitha Godahewa, Christoph Bergmeir, Geoffrey I. Webb, Rob J. Hyndman, and Pablo Montero-Manso. Monash time series forecasting archive. In *NeurIPS Track on Datasets and Benchmarks*, 2021.

Jonathan Goh, Sridhar Adepu, Khurum Nazir Junejo, and Aditya Mathur. A dataset to support research in the design of secure water treatment systems. In *International Conference on Critical Information Infrastructures Security (CRITIS)*, pp. 88–99, 2016.

Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre H. Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Daniel Guo, Mohammad Gheshlaghi Azar, Bilal Piot, Koray Kavukcuoglu, Rémi Munos, and Michal Valko. Bootstrap your own latent: A new approach to self-supervised learning. In *NeurIPS*, 2020.

Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *ICML*, volume 97, pp. 2555–2565. PMLR, 2019.

Danijar Hafner, Kuang-Huei Lee, Ian Fischer, and Pieter Abbeel. Deep hierarchical planning from pixels. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho (eds.), *NeurIPS*, 2022.

Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models, 2024.

Nicklas Hansen, Hao Su, and Xiaolong Wang. Td-mpc2: Scalable, robust world models for continuous control. In *ICLR*, 2024.

Nicklas A Hansen, Hao Su, and Xiaolong Wang. Temporal difference learning for model predictive control. In *ICML*, volume 162 of *Proceedings of Machine Learning Research*, pp. 8387–8406. PMLR, 17–23 Jul 2022.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016.

Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. Momentum contrast for unsupervised visual representation learning. In *CVPR*, 2020.

Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *CVPR*, 2021.

Geoffrey E. Hinton. Training products of experts by minimizing contrastive divergence. volume 14, pp. 1771–1800, Cambridge, MA, USA, August 2002. MIT Press. doi: 10.1162/089976602760128018. URL https://doi.org/10.1162/089976602760128018.

J J Hopfield. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences*, 79(8):2554–2558, 1982. doi: 10.1073/pnas.79.8.2554. URL https://www.pnas.org/doi/abs/10.1073/pnas.79.8.2554.

Herbert Jaeger. Tutorial on training recurrent neural networks, covering bppt, rtrl, ekf and the echo state network approach. *GMD-Forschungszentrum Informationstechnik, 2002.*, 5, 01 2002.

Michael Janner, Yilun Du, Joshua Tenenbaum, and Sergey Levine. Planning with diffusion for flexible behavior synthesis. In *ICML*, 2022.

Wei-Bang Jiang, Li-Ming Zhao, and Bao-Liang Lu. Large brain model for learning generic representations with tractable EEG data. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2405.18765.

Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. Modeling long- and short-term temporal patterns with deep neural networks. In *ACM SIGIR*, pp. 95–104, 2018.

Yann LeCun. A path towards autonomous machine intelligence. *Open Review*, Jun 2022.

Yann LeCun, Sumit Chopra, Raia Hadsell, M Ranzato, and F Huang. A tutorial on energy-based learning. 2006.

Andrew Levy, Robert Platt, and Kate Saenko. Hierarchical reinforcement learning with hindsight. In *ICLR*, 2019.

Lucas Maes, Quentin Le Lidec, Luiz Facury, Nassim Massaudi, Ayush Chaurasia, Francesco Capuano, Richard Gao, Taj Gillin, Dan Haramati, Damien Scieur, Yann LeCun, and Randall Balestriero. stable-worldmodel: A platform for reproducible world modeling research and evaluation. 2026. URL https://arxiv.org/abs/2605.21800.

Ofir Nachum, Shixiang (Shane) Gu, Honglak Lee, and Sergey Levine. Data-efficient hierarchical reinforcement learning. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett (eds.), *NeurIPS*, volume 31. Curran Associates, Inc., 2018.

Adamantios Ntakaris, Martin Magris, Juho Kanniainen, Moncef Gabbouj, and Alexandros Iosifidis. Benchmark dataset for mid-price forecasting of limit order book data with machine learning methods. *Journal of Forecasting*, 37(8):852–866, 2018.

Iyad Obeid and Joseph Picone. The Temple University Hospital EEG data corpus. *Frontiers in Neuroscience*, 10:196, 2016.

Ruben Ohana, Michael McCabe, Lucas Meyer, Rudy Morel, Fruzsina J. Agocs, et al. The well: a large-scale collection of diverse physics simulations for machine learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024. URL https://arxiv.org/abs/2412.00568.

Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel HAZIZA, Francisco Massa, Alaaeldin El-Nouby, Mido Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Herve Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856.

Jack Parker-Holder, Philip Ball, Jake Bruce, Vibhavari Dasagi, Kristian Holsheimer, Christos Kaplanis, Alexandre Moufarek, Guy Scully, Jeremy Shar, Jimmy Shi, Stephen Spencer, Jessica Yung, Michael Dennis, Sultan Kenjeyev, Shangbang Long, Vlad Mnih, Harris Chan, Maxime Gazeau, Bonnie Li, Fabio Pardo, Luyu Wang, Lei Zhang, Frederic Besse, Tim Harley, Anna Mitenkova, Jane Wang, Jeff Clune, Demis Hassabis, Raia Hadsell, Adrian Bolton, Satinder Singh, and Tim Rocktäschel. Genie 2: A large-scale foundation world model. 2024. URL https://deepmind.google/discover/blog/genie-2-a-large-scale-foundation-world-model/.

Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *ICML*, ICML'17, pp. 2778–2787. JMLR.org, 2017.

R. P. Rao and D. H. Ballard. Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. *Nature neuroscience*, 2(1):79–87, January 1999. ISSN 1097-6256. doi: 10.1038/4580. URL http://dx.doi.org/10.1038/4580.

Cory A. Rieth, Ben D. Amsel, Randy Tran, and Maia B. Cook. Additional Tennessee Eastman process simulation data for anomaly detection evaluation. Harvard Dataverse, 2017. doi: 10.7910/DVN/6C3JR1.

Abhinav Saxena, Kai Goebel, Don Simon, and Neil Eklund. Damage propagation modeling for aircraft engine run-to-failure simulation. In *International Conference on Prognostics and Health Management (PHM)*, pp. 1–9, 2008.

Juergen Schmidhuber. On learning to think: Algorithmic information theory for novel combinations of reinforcement learning controllers and recurrent neural world models, 2015. URL https://arxiv.org/abs/1511.09249.

Jurgen Schmidhuber. Making the world differentiable: on using self supervised fully recurrent neural networks for dynamic reinforcement learning and planning in non-stationary environments. *Forschungsberichte, TU Munich*, FKI 126 90:1–26, 1990. URL https://api.semanticscholar.org/CorpusID:28490120.

Ravid Shwartz-Ziv, Randall Balestriero, Kenji Kawaguchi, Tim G. J. Rudner, and Yann LeCun. An information theory perspective on variance-invariance-covariance regularization. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (eds.), *NeurIPS*, volume 36, pp. 33965–33998. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/6b1d4c03391b0aa6ddde0b807a78c950-Paper-Conference.pdf.

Vlad Sobal, Jyothir S V, Siddhartha Jalagam, Nicolas Carion, Kyunghyun Cho, and Yann LeCun. Joint embedding predictive architectures focus on slow features, 2022. URL https://arxiv.org/abs/2211.10831.

Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim Rudner, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models, 02 2025.

Nitish Srivastava, Elman Mansimov, and Ruslan Salakhutdinov. Unsupervised learning of video representations using lstms. In *ICML*, ICML'15, pp. 843–852. JMLR.org, 2015.

Richard S. Sutton. Dyna, an integrated architecture for learning, planning, and reacting. *SIGART Bull.*, 2(4):160–163, July 1991. ISSN 0163-5719. doi: 10.1145/122344.122377. URL https://doi.org/10.1145/122344.122377.

Basile Terver, Randall Balestriero, Megi Dervishi, David Fan, Quentin Garrido, Tushar Nagarajan, Koustuv Sinha, Wancong Zhang, Mike Rabbat, Yann LeCun, and Amir Bar. A lightweight library for energy-based joint-embedding predictive architectures. In *ICLR 2026 Workshop on World Models*, 2026a. URL https://arxiv.org/abs/2602.03604.

Basile Terver, Tsung-Yen Yang, Jean Ponce, Adrien Bardes, and Yann LeCun. What drives success in physical planning with joint-embedding predictive world models?, 2026. URL https://arxiv.org/abs/2512.24497.

Zhan Tong, Yibing Song, Jue Wang, and Limin Wang. Videomae: Masked autoencoders are data-efficient learners for self-supervised video pre-training. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (eds.), *NeurIPS*, volume 35, pp. 10078–10093. Curran Associates, Inc., 2022.

Limin Wang, Bingkun Huang, Zhiyu Zhao, Zhan Tong, Yinan He, Yi Wang, Yali Wang, and Yu Qiao. Videomae v2: Scaling video masked autoencoders with dual masking. In *CVPR*, 2023.

Grady Williams, Andrew Aldrich, and Evangelos Theodorou. Model predictive path integral control using covariance variable importance sampling, 2015.

Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. In *NeurIPS*, 2021.

Haixu Wu, Tengge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. TimesNet: Temporal 2D-variation modeling for general time series analysis. In *ICLR*, 2023.

Renjie Wu and Eamonn J. Keogh. Current time series anomaly detection benchmarks are flawed and are creating the illusion of progress. *IEEE Transactions on Knowledge and Data Engineering*, 35 (3):2421–2429, 2023.

Jure Zbontar, Li Jing, Ishan Misra, Yann LeCun, and Stéphane Deny. Barlow twins: Self-supervised learning via redundancy reduction. In *ICML*, 2021.

Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. Dino-wm: World models on pre-trained visual features enable zero-shot planning, 2024a. URL https://arxiv.org/abs/2411.04983.

Guangyao Zhou, Sivaramakrishnan Swaminathan, Rajkumar Vasudeva Raju, J. Swaroop Guntupalli, Wolfgang Lehrach, Joseph Ortiz, Antoine Dedieu, Miguel Lázaro-Gredilla, and Kevin Murphy. Diffusion model predictive control. *arXiv preprint arXiv:2410.05364*, 2024b.

Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *AAAI*, 2021.